

Laboratorio de Sistemas Operativos

API de Hilos

Práctica No. 4

Fecha de inicio: May. 11 de 2026

Fecha de entrega: Jun. 07 de 2026 (23:59)

Medio de entrega: Moodle

Objetivos

Al finalizar esta práctica de laboratorio, el estudiante será capaz de:

- Aplicar los conceptos teóricos de creación (`pthread_create`) y sincronización (`pthread_join`) de hilos POSIX.
- Implementar la paralelización de una aplicación serial existente (Cálculo de π) para evaluar la mejora en el rendimiento.
- Desarrollar una solución multihilo para un problema de generación de datos (Secuencia de Fibonacci).
- Medir y analizar el impacto del paralelismo mediante el cálculo de métricas de desempeño como **Speedup** y **Eficiencia**.

Prerrequisitos

Se requiere la revisión previa del material teórico de la clase teórica. El estudiante debe estar familiarizado con los siguientes conceptos:

- La distinción conceptual entre *conurrencia* y *paralelismo*.
- El modelo de memoria compartida en programas multihilo.
- Las funciones de la API de Pthreads: `pthread_create`, `pthread_join` y `pthread_exit`.
- Métodos para la transferencia de argumentos a hilos y la gestión de valores de retorno.

1 Paralelización del Cálculo de π

Contexto: Integración Numérica

El archivo `pi.c` calcula el valor de π utilizando un método de integración numérica. El algoritmo se basa en la integral definida:

$$\int_0^1 \frac{4}{1+x^2} dx = \pi$$

El código aproxima el área bajo esta curva dividiéndola en n rectángulos (método de la regla del punto medio) y sumando sus áreas. El bucle `for` en la función `CalcPi` representa el núcleo computacional de la aplicación:

```
1  double CalcPi(int n)
2  {
3      const double fH    = 1.0 / (double) n;
4      double fSum = 0.0;
5      double fX;
6      int i;
7
8      // Bucle principal para paralelizar
9      for (i = 0; i < n; i += 1)
10     {
11         fX = fH * ((double) i + 0.5);
12         fSum += f(fX);
13     }
14     return fH * fSum;
15 }
```

Listing 1: Extracto de `pi.c` — función `CalcPi`

1.1 Actividades a Realizar

1. **Crear `pi.c`:** Termine la implementación del programa `pi.c`
2. **Duplicar el `pi_p.c`:** Genere una copia de `pi.c` y nómbrela `pi_p.c`.
3. **Paralelizar `CalcPi`:** Modifique `pi_p.c` para paralelizar el bucle `for` mediante Pthreads, siguiendo las indicaciones a continuación:
 - **Estrategia (*Data Parallelism*):** La función `main` debe ser modificada para recibir el número de hilos T como argumento de línea de comandos.
 - El rango total de iteraciones (de 0 a $n-1$) debe ser particionado entre los T hilos. Cada hilo será responsable de calcular una **suma parcial** correspondiente a su sub-rango de iteraciones.



- **Gestión de Resultados Parciales:** Se debe evitar el uso de *mutex* dentro del bucle para no introducir alta contención. En su lugar, cada hilo debe calcular su suma en una variable local y retornar dicho valor parcial al hilo principal (p.ej., vía `pthread_join`).
- **Sincronización:** El hilo `main` debe crear los T hilos y sincronizar su finalización mediante `pthread_join`.
- **Resultado Final:** Una vez que todos los hilos han retornado sus sumas parciales, el hilo `main` debe agregar estos resultados y multiplicar la suma total por `fH` para obtener la aproximación final de π .

4. Compilación y Evaluación:

Compile la versión serial con:

```
gcc -o pi_s pi.c -lm
```

Compile la versión paralela con:

```
gcc -o pi_p pi_p.c -lpthread -lm
```

Incorpore instrumentación para la medición de tiempo (p.ej., `GetTime()`) en ambos archivos para medir exclusivamente el tiempo de ejecución de la función `CalcPi`.

2 Generador de Secuencia de Fibonacci

Contexto

El segundo ejercicio consiste en generar la secuencia de Fibonacci en un hilo de trabajo. La secuencia se define recursivamente como:

$$F(-2) = 0, \quad F(-1) = 1, \quad F(n) = F(n-1) + F(n-2) \quad (n \geq 0)$$

2.1 Actividades a Realizar

1. **Crear `fibonacci.c`:** Cree un nuevo archivo fuente con este nombre.

2. **Implementación del Algoritmo:**

- El programa recibirá por línea de comandos el número N de elementos de la secuencia de Fibonacci a generar.
- El hilo `main` será responsable de asignar memoria dinámica (p.ej., `malloc`) para un arreglo compartido de tamaño N .



- Posteriormente, `main` instanciará un único **hilo trabajador**. Se debe pasar a este hilo un puntero al arreglo compartido y el valor de N .
- El **hilo trabajador** calculará los N números de la secuencia y los almacenará secuencialmente en el arreglo compartido.
- El **hilo principal** debe bloquearse (`pthread_join`) hasta que el trabajador complete su ejecución. Tras la finalización del hilo trabajador (confirmada por `pthread_join`), el hilo principal imprimirá la secuencia de Fibonacci contenida en el arreglo compartido.

3. Compilación:

Compile el programa con:

```
gcc -o fibonacci fibonacci.c -lpthread
```

Ejecute para verificar (debe imprimir los primeros 10 números):

```
./fibonacci 10
```

3 Entregable: **analisis.ipynb**

El entregable consiste en un único Jupyter Notebook (`analisis.ipynb`) que contenga los resultados y el análisis de ambas partes. El notebook debe estructurarse de la siguiente manera:

3.1 Sección 1: Análisis de π

1. **Evaluación de T_s (Tiempo Serial):** Reporte el tiempo de ejecución de `./pi_s` con $n = 2\,000\,000\,000$. Este valor será T_s .
2. **Evaluación de T_p (Tiempo Paralelo):** Ejecute `./pi_p` con el mismo n , variando el número de hilos ($N = 1, 2, 4, 8, \dots$ hasta $2 \times$ el número de núcleos de su CPU) y reporte los tiempos $T_p(N)$.
3. **Tabla de Resultados:** Presente una tabla con las métricas de rendimiento calculadas, con el formato mostrado en el Cuadro 1.
4. **Gráfico de Speedup:** Incluya un gráfico de líneas (N Hilos vs. *Speedup*).
5. **Análisis de Resultados:**
 - Realice una comparación entre el rendimiento de $T_p(1)$ y T_s . Explique cualquier discrepancia (*overhead*).

- Analice el Speedup máximo alcanzado. ¿Cómo se compara con el número de núcleos físicos de su sistema?
- Describa la tendencia de la eficiencia a medida que N incrementa y explique las causas de dicho comportamiento.

Tab. 1: Métricas de rendimiento para el cálculo paralelo de π

N (Hilos)	T_p (segundos)	Speedup (T_s/T_p)	Eficiencia (Speedup/ N)
1	...	...	...
2	...	...	...
4	...	...	...
8	...	...	...
16	...	...	...

3.2 Sección 2: Análisis de Fibonacci

1. **Resultados de Ejecución:** Incluya la salida de su programa `./fibonacci 15`.
2. **Análisis del Diseño:**
 - Implementar el cálculo de la serie de Fibonacci sin el uso de hilos y probarlo (en tiempo) para N grande (más de $100e3$ valores).
 - Describa el mecanismo utilizado para transferir datos (el puntero al arreglo y N) del hilo principal al hilo trabajador.
 - Explique el rol de `pthread_join` como mecanismo de sincronización en este problema, asegurando que `main` no acceda a los resultados antes de que sean generados.

Entrega y evaluación

Incluya un enlace a un repositorio de github con:

1. Informe donde se explica claramente la solución (README.md). El informe debe contar con las siguientes secciones:
 - (a) Nombres completos de los integrantes, correos y números de documento.
 - (b) Documentación de todas las funciones desarrolladas en el código.
 - (c) Problemas presentados durante el desarrollo de la práctica y sus soluciones.
 - (d) Pruebas realizadas a los programas que verificaron su funcionalidad.
 - (e) Un enlace a un video de 10 minutos donde se sustente el desarrollo.
 - (f) Manifiesto de transparencia: En que puntos se apoyaron de la IA generativa.

- ## Contenido del video:

- Trabajo a realizar en parejas, o en caso excepcional grupo de 3 personas, evitar trabajar solo.
Entregar un solo repositorio por grupo.

- Remzi H. Arpaci-Dusseau & Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*.
 - **Capítulo 26:** Threads Intro. Enlace
 - **Capítulo 27:** Thread API. Enlace
 - **Diapositivas de apoyo:** Interlude: Thread_API. Enlace

6